# Ps 01

Erik An

*<2026-03-06 Fri>*

## Contents

**1 Problem 1** **DONE** **4**

1.1 Task . . . . . 4

1.2 Solution . . . . . 4

**2 Problem 2** **DONE** **6**

2.1 Task . . . . . 6

2.2 Solution . . . . . 7

**3 Problem 3** **DONE** **7**

3.1 Task . . . . . 7

3.2 Solution . . . . . 7

**4 Problem 4** **DONE** **8**

4.1 Task . . . . . 8

4.2 Solution . . . . . 8

**5 Problem 5** **DONE** **9**

5.1 Task . . . . . 9

5.2 Solution . . . . . 9

**6 Problem 6** **DONE** **9**

6.1 Task . . . . . 9
6.2 Solution . . . . . 10

**7 Problem 7 DONE 10**

7.1 Task . . . . . 10
7.2 Solution . . . . . 10

**8 Problem 8 DONE 11**

8.1 Task . . . . . 11
8.2 Solution . . . . . 11

**9 Problem 9 DONE 11**

9.1 Task . . . . . 11
9.2 Solution . . . . . 12
9.2.1 (a) . . . . . 12
9.2.2 (b) . . . . . 12
9.2.3 (c) . . . . . 12
9.2.4 (d) . . . . . 13
9.2.5 (e) . . . . . 13
9.2.6 (f) . . . . . 13

**10 Problem 10 DONE 14**

10.1 Task . . . . . 14
10.2 Solution . . . . . 14

**11 Problem 11 DONE 15**

11.1 Task . . . . . 15
11.2 Solution . . . . . 15

**12 Problem 12 DONE 15**

12.1 Task . . . . . 15
12.2 Solution . . . . . 15

**13 Problem 13 DONE 16**

13.1 Task . . . . . 16
13.2 Solution . . . . . 16
13.2.1 (a) . . . . . 16
13.2.2 (b) . . . . . 17
13.2.3 (c) . . . . . 17

**14 Problem 14 SKIP 17**

14.1 Task . . . . . 17
14.2 Solution . . . . . 18

**15 Problem 15 DONE 18**

15.1 Task . . . . . 18
15.2 Solution . . . . . 19

**16 Problem 16 DONE 19**

16.1 Task . . . . . 19
16.2 Solution . . . . . 20

**17 Problem 17 DONE 20**

17.1 Task . . . . . 20
17.2 Solution . . . . . 21

# 1 Problem 1 DONE

## 1.1 Task

Assume that your computer can execute $10^9$ operations per second. For each of the following numbers of operations $f(n)$ and the given time limits calculate the (approximal) maximal size $n$ of your problem that can be solved.

| $f(n)$ | 1 $\mu s$ | 1 $ms$ | 1 $s$ | 1 $min$ | 1 $hour$ | 1 $day$ | 1 $week$ | 30 $days$ | 1 $year$ | 1000 $years$ |
|---|---|---|---|---|---|---|---|---|---|---|
| $\lg \lg n$ | | | | | | | | | | |
| $\lg n$ | | | | | | | | | | |
| $\sqrt{n}$ | | | | | | | | | | |
| $n$ | | | | | | | | | | |
| $n \lg n$ | | | | | | | | | | |
| $n^2$ | | | | | | | | | | |
| $n^3$ | | | | | | | | | | |
| $n^4$ | | | | | | | | | | |
| $2^n$ | | | | | | | | | | |
| $n!$ | | | | | | | | | | |

## 1.2 Solution

```
import Printf: @sprintf
using Printf

const OPS_PER_SEC = 1e9

time_labels  = ["1 s", "1 ms", "1 s", "1 min", "1 hour", "1 day", "1 week", "30 days", "1 year",
time_seconds = [1e-6, 1e-3, 1.0, 60.0, 3600.0, 86400.0, 604800.0, 2.592e6, 3.1536e7, 3.1536e10]

#  Inverse functions

# lg lg n = ops  →  n = 2^(2^ops)
function inv_lglg(ops)
    return "2^2^$(@sprintf("%.3g", ops))"
end

# lg n = ops  →  n = 2^ops = 10^(ops * log10(2))
function inv_lg(ops)
    log10_val = ops * log10(2)
    return "10^$(@sprintf("%.3g", log10_val))"
end

# n = ops  →  n = ops²
inv_sqrt(ops) = @sprintf("%.3g", ops^2)
```

```
# n = ops
inv_n(ops) = @sprintf("%.3g", ops)

# n lg n = ops  →  binary search
function inv_nlgn(ops)
    ops < 1 && return "0"
    lo, hi = 1.0, ops
    for _ in 1:100
        mid = (lo + hi) / 2
        val = mid * log2(mid)
        val < ops ? (lo = mid) : (hi = mid)
    end
    return @sprintf("%.3g", lo)
end

# n² = ops  →  n = ops
inv_n2(ops) = @sprintf("%.3g", sqrt(ops))

# n³ = ops  →  n = ops^(1/3)
inv_n3(ops) = @sprintf("%.3g", ops^(1/3))

# n = ops  →  n = ops^(1/4)
inv_n4(ops) = @sprintf("%.3g", ops^(1/4))

# 2 = ops  →  n = floor(log(ops))
function inv_2n(ops)
    ops < 1 && return "0"
    return @sprintf("%.3g", floor(log2(ops)))
end

# n! = ops  →  largest n where n!  ops
function inv_nfact(ops)
    ops < 1 && return "0"
    n = BigInt(1)
    fact = BigInt(1)
    ops_big = BigInt(floor(ops))

    while true
        next_fact = fact * (n + 1)
        if next_fact > ops_big
            break
        end
        n += 1
        fact = next_fact
    end
    return string(n)
end
```

```
#  Build table

func_labels = ["lg lg n", "lg n", "n", "n", "n lg n", "n²", "n³", "n", "2", "n!"]
inv_funcs   = [inv_lglg, inv_lg, inv_sqrt, inv_n, inv_nlgn, inv_n2, inv_n3, inv_n4, inv_2n, inv_

results = Matrix{String}(undef, length(func_labels), length(time_labels))

for (j, t) in enumerate(time_seconds)
    ops = t * OPS_PER_SEC
    for (i, f) in enumerate(inv_funcs)
        results[i, j] = f(ops)
    end
end

#  Pretty print (Org/Markdown Format)

header = "| \$f(n)\$ | " * join(time_labels, " | ") * " |"
separator = "|---|$(join(fill("---", length(time_labels)), "|"))|"

println(header)
println(separator)
for i in 1:length(func_labels)
    row = "| \$$(func_labels[i])\$ | " * join(results[i, :], " | ") * " |"
    println(row)
end
```

| $f(n)$ | 1 s | 1 ms | 1 s | 1 min | 1 hour | 1 day | 1 week | 30 days | |
|---|---|---|---|---|---|---|---|---|---|
| $lglgn$ | $2^{21e}$+03 | $2^{21e}$+06 | $2^{21e}$+09 | $2^{26e}$+10 | $2^{23.6e}$+12 | $2^{28.64e}$+13 | $2^{26.05e}$+14 | $2^{22.59e}$+15 | $2^{23.}$ |
| $lgn$ | $10^{301}$ | $10^{3.01e}$+05 | $10^{3.01e}$+08 | $10^{1.81e}$+10 | $10^{1.08e}$+12 | $10^{2.6e}$+13 | $10^{1.82e}$+14 | $10^{7.8e}$+14 | $10^{9.}$ |
| $n$ | 1e+06 | 1e+12 | 1e+18 | 3.6e+21 | 1.3e+25 | 7.46e+27 | 3.66e+29 | 6.72e+30 | 9.9 |
| $n$ | 1e+03 | 1e+06 | 1e+09 | 6e+10 | 3.6e+12 | 8.64e+13 | 6.05e+14 | 2.59e+15 | 3.1 |
| $nlgn$ | 140 | 6.27e+04 | 3.96e+07 | 1.94e+09 | 9.86e+10 | 2.11e+12 | 1.39e+13 | 5.67e+13 | 6.4 |
| $n$ | 31.6 | 1e+03 | 3.16e+04 | 2.45e+05 | 1.9e+06 | 9.3e+06 | 2.46e+07 | 5.09e+07 | 1.7 |
| $n$ | 10 | 100 | 1e+03 | 3.91e+03 | 1.53e+04 | 4.42e+04 | 8.46e+04 | 1.37e+05 | 3.1 |
| $n$ | 5.62 | 31.6 | 178 | 495 | 1.38e+03 | 3.05e+03 | 4.96e+03 | 7.14e+03 | 1.3 |
| 2 | 9 | 19 | 29 | 35 | 41 | 46 | 49 | 51 | |
| $n!$ | 6 | 9 | 12 | 13 | 15 | 16 | 17 | 17 | |

# 2 Problem 2 DONE

## 2.1 Task

Consider the algorithm SUM ARRAY from page 25 of the lecture book:

1: procedure SUM ARRAY 2: sum = 0 3: for i = 1 to n do 4: sum = sum + A[i] 5: end for 6: end procedure

It should solve the problem to sum the elements of the array $A$. Prove that the algorithm is correct by using a loop invariant.

### 2.2 Solution

**Loop invariant:** At the start of each iteration with index $i$, $sum = A[1] + A[2] + \cdots + A[i-1]$.

**Initialization (i = 1):** $sum = 0$, which equals the empty sum $A[1] + \cdots + A[0]$ (a sum of zero terms). This holds.

**Maintenance:** Assume at the start of iteration $i$ we have $sum = A[1] + \cdots + A[i-1]$. The loop body executes $sum = sum + A[i]$, so after the body, $sum = A[1] + \cdots + A[i]$. When $i$ increments to $i+1$, we're at the start of the next iteration and $sum = A[1] + \cdots + A[i+1-1]$. The invariant holds for the next iteration.

**Termination:** The loop ends when $i = n+1$. Substituting into the invariant: $sum = A[1] + \cdots + A[n]$, which is exactly what we wanted to prove.

## 3 Problem 3 DONE

### 3.1 Task

We define the searching problem as follows: Input: A sorted sequence of $n$ numbers $(a_1, a_2, \ldots, a_n)$ with $a_i < a_{i+1}$ for all $i < n$ provided in array $A$, a value $x$. Output: An index $i$ such that $x = A[i]$ and if $x$ does not appear in $A$, return the largest index $i$ such that $A[i] < x$. If no such index exists, return 0. Write pseudocode for linear search that scans the array from the beginning to the end to determine $i$. Implement the procedure in Julia.

### 3.2 Solution

```
function linear_search(array, x)
    n = length(array)
    i = 1
    while (i <= n) && (array[i] <= x)
        i += 1
    end
    return i - 1
end

function main()
```

```
    A = [1, 2, 3, 4, 6, 7, 8, 9]
    println(linear_search(A, 4))
    println(linear_search(A, 5))
    println(linear_search(A, 0))
    println(linear_search(A, 10))
end

main()
```

```
linear_search (generic function with 1 method)
main (generic function with 1 method)
4
4
0
8
```

# 4 Problem 4 DONE

## 4.1 Task

Using a loop invariant, prove that your algorithm from Exercise 3 is correct. Make sure that your loop invariant fulfills the three necessary properties. Give an asymptotic running time using the Landau symbols for the worst-case.

## 4.2 Solution

**Loop invariant:** $\forall j : 1 \leq j < i, A[j] \leq x$

**Initialization (i = 1):** The range $j = 1, \cdots, 0$ is empty, so the statement "for all $j < 1$, $A[j] \leq x$" holds vacuously.

**Maintenance:** Assume the invariant holds at the start of iteration $i$, meaning $A[1], \ldots, A[i-1]$ are all $\leq x$. The loop condition checks that $i \leq n$ and $A[i] \leq x$. Since we enter the body, we know $A[i] \leq x$. After incrementing $i$ to $i+1$, we now know $A[1], \ldots, A[(i+1)-1]$ are all $\leq x$. The invariant holds for the next iteration.

**Termination:** The loop exits when either $i > n$ or $A[i] > x$. In both cases the invariant tells us $A[1], \ldots, A[i-1]$ are all $\leq x$. Since the array is sorted, $i-1$ is the largest index with $A[i-1] \leq x$. If $A[i-1] = x$, then $i-1$ is the index of $x$. If $A[i-1] < x$ (or $i = 1$), then $x$ is not in the array and $i-1$ is the largest index with $A[i-1] < x$ (or 0 if no such index exists). This matches the required output.

**Worst-case running time:** The loop executes at most $n$ iterations (when all elements are $\leq x$), and each iteration does constant work, so the worst-case running time is $\mathcal{O}(n)$.

# 5 Problem 5 DONE

## 5.1 Task

Let an array $A[1:n]$ of numbers be given. Consider the following algorithm for solving the sorting problem: Find the smallest element of $A[1:n]$ and then exchange it with the first element. Afterwards, find the smallest element in $A[2:n]$ and exchange it with the second element, and so on. This algorithm is called selection sort. Write pseudocode for this algorithm and implement it in Julia.

## 5.2 Solution

```
function selection_sort(array)
    n = length(array)

    for i in 1:n
        j = argmin(array[i:end]) + i - 1
        array[i], array[j] = array[j], array[i]
    end
    return array
end

function main()
    array = [9, 8, 7, 6, 5, 4, 3, 2, 1]
    println(selection_sort(array))
end

main()
```

```
selection_sort (generic function with 1 method)
main (generic function with 1 method)
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

# 6 Problem 6 DONE

## 6.1 Task

Take the algorithm selection sort from Exercise 5 and find a loop invariant for that algorithm. Prove your assumption.

### 6.2 Solution

**Loop invariant:** At the start of each iteration with index $i$, the subarray $A[1], \ldots, A[i-1]$ is sorted and contains the $i-1$ smallest elements of the original array.

**Initialization (i = 1):** The subarray $A[1], \cdots, A[0]$ is empty. An empty subarray is trivially sorted and trivially contains the 0 smallest elements.

**Maintenance:** Assume at the start of iteration $i$, $A[1], \cdots, A[i-1]$ is sorted and contains the $i-1$ smallest elements. The algorithm finds the minimum of $A[i], \cdots, A[n]$ and swaps it into position $i$. Since $A[1], \cdots, A[i-1]$ already contains the $i-1$ smallest elements, the minimum of the remaining subarray is the $i$-th smallest element overall. After the swap, $A[i]$ holds this value, so $A[1], \cdots, A[i]$ is sorted and contains the $i$ smallest elements. When $i$ increments, the invariant holds for the next iteration.

**Termination:** The loop ends when $i = n+1$. The invariant gives us that $A[1], \cdots, A[n]$ is sorted and contains the $n$ smallest elements, which means the entire array is sorted.

## 7 Problem 7 DONE

### 7.1 Task

> Analyse your algorithm from Exercise 5 and determine its best- and worst-case running times. Provide asymptotic running times using the Landau symbols ($O$, $\Omega$, or $\Theta$), whichever fit best.

### 7.2 Solution

The outer loop runs from $i = 1$ to $n$. In each iteration, finding the minimum of $A[i \ldots n]$ requires $n - i$ comparisons, and the swap is constant time. This work is the same regardless of the input.

Total comparisons:

$$\sum_{i=1}^{n}(n-i) = \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

**Upper bound ($O$):** For $n \geq 1$, we have $\frac{n^2}{2} - \frac{n}{2} \leq \frac{n^2}{2} \leq n^2$. Choose $c = 1$, $n_0 = 1$. So $f(n) = O(n^2)$.

**Lower bound ($\Omega$):** For $n \geq 2$, we have $\frac{n}{2} \leq \frac{n^2}{4}$, so $\frac{n^2}{2} - \frac{n}{2} \geq \frac{n^2}{2} - \frac{n^2}{4} = \frac{n^2}{4}$. Choose $c = \frac{1}{4}$, $n_0 = 2$. So $f(n) = \Omega(n^2)$.

Since $f(n) = O(n^2)$ and $f(n) = \Omega(n^2)$, we conclude $f(n) = \Theta(n^2)$.

Since selection sort always performs the same number of comparisons regardless of the input, the best-case and worst-case running times are both $\Theta(n^2)$.

# 8 Problem 8 DONE

## 8.1 Task

Use mathematical induction to show that when $n = 2^k$ for $k \geq 1$, the solution of the recurrence

$T(n) = \begin{cases} 2 & n = 2 \\ 2T\left(\frac{n}{2}\right) + n & n > 2 \end{cases}$

is $T(n) = n \lg n$.

## 8.2 Solution

- Base case

  $k = 1$, so $n = 2$. Hence $T(2) = 2 \cdot \lg(2) = 2 \cdot 1 = 2$. Base holds.

- Inductive step

  Assume $T(2^k) = 2^k \cdot k$ (since $\lg(2^k) = k$).

  Now show that $T(2^{k+1}) = 2^{k+1} \cdot (k+1)$.

  Hence,

$$\begin{aligned} T(2^{k+1}) &= 2 \cdot T(\frac{2^{k+1}}{2}) + 2^{k+1} \\ &= 2 \cdot T(2^k) + 2^{k+1} \\ &= 2 \cdot (2^k \cdot k) + 2^{k+1} \\ &= 2^{k+1} \cdot k + 2^{k+1} \\ &= 2^{k+1}(k+1) \end{aligned}$$

Hence, $T(n) = n \lg(n)$

# 9 Problem 9 DONE

## 9.1 Task

(a) Express $123n^2 + 345n - 148$ as $\Theta$ of a simple expression.

(b) Is $2^{n+1} = O(2^n)$?

(c) Is $2^{2n} = O(2^n)$?

(d) Is $2^n = O(2^{2n})$?

(e) Is $2^{n+1} = o(2^n)$?

(f) Is $2^n = o(2^{2n})$?

## 9.2 Solution

### 9.2.1 (a)

- Upper bound

$$123n^2 + 345n - 148 \leq 123n^2 + 345n^2 = 468n^2$$

  Choose $c = 468, n_0 = 1$. So, $f(n) = O(n^2)$

- Lower bound

$$123n^2 + 345n - 148 \geq 123n^2 + 345 - 148 \geq 123n^2$$

  Choose $c = 123, n_0 = 1$. So $f(n) = \Omega(n^2)$

Since $f(n) = O(n^2)$ and $f(n) = \Omega(n^2)$, we conclude $f(n) = \Theta(n^2)$.

### 9.2.2 (b)

$$2^{n+1} = 2 \cdot 2^n$$

Choose $c = 2, n \geq 1$. $f(n) = O(2^n)$

### 9.2.3 (c)

Assume $2^{2n} = O(2^n)$. Hence,

$$\begin{aligned} 2^{2n} &\leq c \cdot 2^n \\ 2^n \cdot 2^n &\leq c \cdot 2^n \\ 2^n &\leq c \end{aligned}$$

But $2^n$ grows without a bound, hence, no constant $c$ can satisfy this for all sufficiently large $n$.

Thus,

$$2^{2n} \neq O(2^n)$$

### 9.2.4 (d)

Assume $2^n = O(2^{2n})$. Let $c = 1$, hence,

$$\begin{aligned} 2^n &\leq 1 \cdot 2^n \cdot 2^n \\ 1 &\leq 2^n \end{aligned}$$

Which is true $\forall n \geq 1$.

### 9.2.5 (e)

$2^{n+1} = o(2^n)$ iff:

$$\begin{aligned} \lim_{n\to\infty} \frac{2^{n+1}}{2^n} &= 0 \\ \lim_{n\to\infty} \frac{2 \cdot 2^n}{2^n} &= 0 \\ 2 \cdot \lim_{n\to\infty} \frac{2^n}{2^n} &= 0 \\ 2 \cdot 1 &= 0 \\ 2 &\neq 0 \end{aligned}$$

Hence,

$$2^{n+1} \neq o(2^n)$$

### 9.2.6 (f)

$2^n = o(2^{2n})$ iff:

$$
\begin{aligned}
\lim_{n\to\infty} \frac{2^n}{2^{2n}} &= 0 \\
\lim_{n\to\infty} \frac{2^n}{2^n \cdot 2^n} &= 0 \\
\lim_{n\to\infty} \frac{1}{2^n} &= 0 \\
0 &= 0
\end{aligned}
$$

Hence,

$$2^n = o(2^{2n})$$

# 10 Problem 10 DONE

## 10.1 Task

Consider sequences $f(n)$, $g(n)$ and $h(n)$. Assume that $f(n) = \Theta(g(n))$ and that $g(n) = \Theta(h(n))$. Show that then $f(n) = \Theta(h(n))$.

## 10.2 Solution

Assume $f(n) = \Theta(g(n)), g(n) = \Theta(h(n))$. Show $f(n) = \Theta(h(n))$.

From $f(n) = \Theta(g(n))$: $\exists c_1, c_2 > 0$, $n_1 : c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n), \forall n \geq n_1$.

From $g(n) = \Theta(h(n))$: $\exists c_3, c_4 > 0$, $n_2 : c_3 \cdot h(n) \leq g(n) \leq c_4 \cdot h(n), \forall n \geq n_2$.

Let $n_0 = \max(n_1, n_2)$. Then for all $n \geq n_0$:

$$
\begin{aligned}
c_1 \cdot g(n) \leq & f(n) \leq c_2 \cdot g(n) \\
c_3 \cdot h(n) \leq & g(n) \leq c_4 \cdot h(n) \\
c_1 \cdot c_3 \cdot h(n) \leq c_1 \cdot g(n) \leq & f(n) \leq c_2 \cdot g(n) \leq c_2 \cdot c_4 \cdot h(n)
\end{aligned}
$$

Let $c_5 = c_1 \cdot c_3$, $c_6 = c_2 \cdot c_4$. Then:

$$c_5 \cdot h(n) \leq f(n) \leq c_6 \cdot h(n), \quad \forall n \geq n_0$$

Thus, $f(n) = \Theta(h(n))$. ■

# 11 Problem 11 DONE

## 11.1 Task

For sequences $f(n)$, $g(n)$, $h(n)$, and $k(n)$ prove that $f(n) = \Omega(h(n))$ and $g(n) = \omega(k(n))$ implies $f(n) + g(n) = \Omega(h(n) + k(n))$.

## 11.2 Solution

Assume $f(n) = \Omega(h(n))$, $g(n) = \omega(k(n))$. Show $f(n) + g(n) = \Omega(h(n) + k(n))$.

$$\Omega(h(n)) = \{f(n) : \exists c_1, n_1 > 0 : 0 \leq c_1 \cdot h(n) \leq f(n), \forall n \geq n_1\}$$

$$\omega(k(n)) = \{g(n) : \forall c_2 > 0, \exists n_2 > 0 : 0 \leq c_2 \cdot k(n) < g(n), \forall n \geq n_2\}$$

Let $n_0 = \max(n_1, n_2)$ and $c_2 = c_1$. Then $\forall n \geq n_0$:

$$\begin{gathered} 0 \leq c_1 \cdot h(n) \leq f(n) \\ 0 \leq c_1 \cdot k(n) < g(n) \\ 0 \leq c_1 \cdot (h(n) + k(n)) \leq f(n) + g(n) \end{gathered}$$

Hence $f(n) + g(n) = \Omega(h(n) + k(n))$. ■

# 12 Problem 12 DONE

## 12.1 Task

Let $f(n) = O(g(n))$ and $g(n) = o(h(n))$. Prove that $f(n) = o(h(n))$.

## 12.2 Solution

Assume $f(n) = O(g(n))$ and $g(n) = o(h(n))$. Show $f(n) = o(h(n))$.

$$O(g(n)) = \{f(n) : \exists c_1, n_1 > 0 : 0 \leq f(n) \leq c_1 \cdot g(n), \forall n \geq n_1\}$$

$$o(h(n)) = \{g(n) : \forall c_2 > 0, \exists n_2 > 0 : 0 \leq g(n) < c_2 \cdot h(n), \forall n \geq n_2\}$$

Let $c > 0$ be arbitrary. Choose $c_2 = \frac{c}{c_1}$ and $n_0 = \max(n_1, n_2)$. Then $\forall n \geq n_0$:

$$\begin{aligned} 0 \leq f(n) &\leq c_1 \cdot g(n) \\ &< c_1 \cdot c_2 \cdot h(n) \\ &< c_1 \cdot \frac{c}{c_1} \cdot h(n) \\ &< c \cdot h(n) \end{aligned}$$

Hence $f(n) = o(h(n))$. ■

# 13 Problem 13 DONE

## 13.1 Task

Use Stirling's formula

$$n! = \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right).$$

to show that

(a) $n! = o(n^n)$,

(b) $n! = \omega(2^n)$,

(c) $\lg(n!) = \Theta(n \lg n)$.

## 13.2 Solution

### 13.2.1 (a)

$$\begin{aligned} \lim_{n\to\infty} \frac{\sqrt{2\pi n}\left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right)}{n^n} &= \lim_{n\to\infty} \sqrt{2\pi n} \cdot \frac{1}{e^n} \cdot \left(1 + \Theta\left(\frac{1}{n}\right)\right) \\ &= \lim_{n\to\infty} \sqrt{2\pi n} \cdot \frac{1}{e^n} \cdot 1 \\ &= 0 \end{aligned}$$

since $\sqrt{2\pi n}$ grows polynomially while $e^n$ grows exponentially.

Hence $n! = o(n^n)$. ■

#### 13.2.2 (b)

$$\begin{aligned}
\lim_{n\to\infty} \frac{\sqrt{2\pi n}\left(\frac{n}{e}\right)^n \left(1+\Theta\left(\frac{1}{n}\right)\right)}{2^n} &= \lim_{n\to\infty} \sqrt{2\pi n} \cdot \left(\frac{n}{2e}\right)^n \cdot \left(1+\Theta\left(\frac{1}{n}\right)\right) \\
&= \lim_{n\to\infty} \sqrt{2\pi n} \cdot \left(\frac{n}{2e}\right)^n \cdot 1 \\
&= \infty
\end{aligned}$$

Hence, $n! = \omega(2^n)$. ■

#### 13.2.3 (c)

$$\begin{aligned}
\lg(n!) &= \lg\left(\sqrt{2\pi n}\right) + n\lg n - n\lg e + \lg\left(1+\Theta\left(\frac{1}{n}\right)\right) \\
\frac{\lg(n!)}{n\lg n} &= \frac{\frac{1}{2}\lg(2\pi n)}{n\lg n} + 1 - \frac{\lg e}{\lg n} + \frac{\lg\left(1+\Theta\left(\frac{1}{n}\right)\right)}{n\lg n} \\
\lim_{n\to\infty} \frac{\lg(n!)}{n\lg n} &= 0 + 1 - 0 + 0 = 1
\end{aligned}$$

Since the limit is a positive constant, $\lg(n!) = \Theta(n\lg n)$. ■

## 14 Problem 14 SKIP

### 14.1 Task

We define the iterated logarithm function

$$\lg^*(n) := \min\{i \in \mathbb{N} \mid \lg^{(i)}(n) \leq 1\}$$

where $\lg^{(1)}(x) := \lg x$, $\lg^{(k+1)}(x) := \lg(\lg^{(k)}(x))$. This function grows extremely slowly. Order the following functions by asymptotic growth:

| # | Function |
|---|---|
| 1 | $\lg(\lg^*(n))$ |
| 2 | $2^{\lg^*(n)}$ |
| 3 | $\sqrt{2}^{\lg n}$ |
| 4 | $n^2$ |
| 5 | $n!$ |
| 6 | $(\lceil \lg n \rceil)!$ |
| 7 | $\left(\frac{3}{2}\right)^n$ |
| 8 | $n^3$ |
| 9 | $\lg^2 n$ |
| 10 | $\lg(n!)$ |
| 11 | $2^{2^n}$ |
| 12 | $n^{1/\lg n}$ |
| 13 | $\lg \lg n$ |
| 14 | $\lg^* n$ |
| 15 | $n \cdot 2^n$ |
| 16 | $n^{\lg \lg n}$ |
| 17 | $\lg n$ |
| 18 | $1$ |
| 19 | $2^{\lg n}$ |
| 20 | $(\lg n)^{\lg n}$ |
| 21 | $e^n$ |
| 22 | $4^{\lg n}$ |
| 23 | $(n+1)!$ |
| 24 | $\sqrt{\lg n}$ |
| 25 | $\lg^*(\lg n)$ |
| 26 | $2^{\sqrt{2 \lg n}}$ |
| 27 | $n$ |
| 28 | $2^n$ |
| 29 | $n \lg n$ |
| 30 | $2^{2^{n+1}}$ |

### 14.2 Solution

## 15 Problem 15 DONE

### 15.1 Task

Use the substitution method to prove that the recursion

$$T(n) = 4T\left(\frac{n}{2}\right) + n$$

implies $T(n) = \Theta(n^2)$.

Hint: Subtract a lower order term.

### 15.2 Solution

- Upper bound: $T(n) = O(n^2)$

  Trying $T(n) \leq cn^2$ directly gives $cn^2 + n$, which doesn't close because the $+n$ term cannot be absorbed. To fix this, subtract a lower-order term from the hypothesis.

  - Inductive hypothesis: assume $T(k) \leq ck^2 - dk$ for all $k < n$.
  - Base case: $T(1) = a$. We need $a \leq c \cdot 1 - d \cdot 1 = c - d$. With $d = 1$, this requires $c \geq a + 1$, which is satisfiable.
  - Substitute:

$$\begin{aligned} T(n) &\leq 4\left(c \cdot \frac{n^2}{4} - d \cdot \frac{n}{2}\right) + n \\ &= cn^2 - 2dn + n \\ &= cn^2 - dn - (dn - n) \\ &\leq cn^2 - dn \end{aligned}$$

    Last step holds when $dn - n \geq 0$, i.e. $d \geq 1$. Pick $d = 1$.
    Since $cn^2 - dn \leq cn^2$, this establishes $T(n) = O(n^2)$.

- Lower bound: $T(n) = \Omega(n^2)$

  - Inductive hypothesis: assume $T(k) \geq ck^2$ for all $k < n$.
  - Base case: $T(1) = a$. We need $a \geq c \cdot 1 = c$, so choose $c \leq a$.
  - Substitute:

$$\begin{aligned} T(n) &\geq 4\left(c \cdot \frac{n^2}{4}\right) + n \\ &= cn^2 + n \\ &\geq cn^2 \end{aligned}$$

    Holds trivially since $n > 0$.

- Conclusion

  Since $T(n) = O(n^2)$ and $T(n) = \Omega(n^2)$, we have $T(n) = \Theta(n^2)$. ■

## 16 Problem 16 DONE

### 16.1 Task

For the recurrence

$$T(n) = 4T\left(\frac{n}{3}\right) + n$$

sketch the recursion tree and derive a guess for an asymptotic upper bound of its solution. Then use induction to prove your guess.

### 16.2 Solution

- Deriving the guess via recursion tree

  The recurrence $T(n) = 4T(n/3) + n$ spawns 4 subproblems of size $n/3$ at each level. At level $k$ there are $4^k$ nodes, each contributing $n/3^k$ work, so the total work at level $k$ is $(4/3)^k \cdot n$. The tree has depth $\log_3 n$. Since the ratio $4/3 > 1$, the geometric series is dominated by its last term:

  $$4^{\log_3 n} = n^{\log_3 4}$$

  This suggests $T(n) = O(n^{\log_3 4})$.

- Proof by substitution

  - Inductive hypothesis: assume $T(k) \leq ck^{\log_3 4} - dk$ for all $k < n$.
  - Base case: $T(1) = a$. We need $a \leq c \cdot 1 - d \cdot 1 = c - d$. With $d = 3$, this requires $c \geq a + 3$, which is satisfiable.
  - Substitute:

  $$\begin{aligned} T(n) &\leq 4\left(\frac{n^{\log_3 4}}{4} - d \cdot \frac{n}{3}\right) + n \\ &= cn^{\log_3 4} - \frac{4dn}{3} + n \\ &= cn^{\log_3 4} - dn - \left(\frac{dn}{3} - n\right) \\ &\leq cn^{\log_3 4} - dn \end{aligned}$$

  Last step holds when $\frac{dn}{3} - n \geq 0$, i.e. $d \geq 3$. Pick $d = 3$.

Since $cn^{\log_3 4} - dn \leq cn^{\log_3 4}$, this establishes $T(n) = O(n^{\log_3 4})$. ■

## 17 Problem 17 DONE

### 17.1 Task

Consider the recursion describing Strassen's algorithm for matrix multiplication. Using the Master Theorem, prove that its running time $T(n)$ satisfies $T(n) = O(n^{\lg 7})$.

## 17.2 Solution

Strassen's algorithm satisfies the recurrence $T(n) = 7\,T(n/2) + \Theta(n^2)$.

Apply the Master Theorem with $a = 7$, $b = 2$, $f(n) = \Theta(n^2)$.

We have $n^{\log_b a} = n^{\log_2 7} = n^{\lg 7}$. Since $\lg 7 \approx 2.81$, the cost $f(n) = n^2$ is polynomially smaller than $n^{\lg 7}$:

$$f(n) = O(n^{\lg 7 - \epsilon}) \quad \text{with } \epsilon \approx 0.81$$

This is Case 1 of the Master Theorem, which gives $T(n) = \Theta(n^{\lg 7})$, and therefore $T(n) = O(n^{\lg 7})$.
■